1. Greedy Best-First Search

```
import heapq

# Use case: Finding a route using estimated distance to the goal.
graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('D', 5), ('E', 1)],
    'C': [('F', 3), ('G', 6)],
    'D': [], 'E': [('H', 2)], 'F': [('H', 4)],
    'G': [('H', 2)], 'H': []
}

# Heuristic estimate from each node to the goal.
heuristic = {
    'A': 7, 'B': 5, 'C': 4, 'D': 6,
    'E': 2, 'F': 3, 'G': 2, 'H': 0
}

def greedy_best_first_search(start, goal):
    priority_queue = [(heuristic[start], start)]
    visited = set()
    parent = {start: None}

    while priority_queue:
        _, node = heapq.heappop(priority_queue)

        if node in visited:
            continue
        visited.add(node)

        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]

        for neighbor, cost in graph[node]:
            if neighbor not in visited and neighbor not in parent:
                parent[neighbor] = node
                heapq.heappush(
                    priority_queue, (heuristic[neighbor], neighbor)
                )

    return None

path = greedy_best_first_search('A', 'H')
print("Path:", path if path else "No path found")
```

2. A* Search

```
import heapq

# Use case: Finding the least-cost path between locations.
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1)],
    'D': []
}

heuristic = {'A': 4, 'B': 3, 'C': 1, 'D': 0}

def a_star_search(start, goal):
    # Entries contain (estimated total cost, current cost, node).
    queue = [(heuristic[start], 0, start)]
    parent = {start: None}
    best_cost = {start: 0}

    while queue:
        estimated_total, cost, node = heapq.heappop(queue)

        if cost != best_cost[node]:
            continue

        if node == goal:
            path = []
            current = goal
```

```
            while current is not None:
                path.append(current)
                current = parent[current]
            return path[::-1], cost

        for neighbor, edge_cost in graph[node]:
            new_cost = cost + edge_cost
            if new_cost < best_cost.get(neighbor, float("inf")):
                best_cost[neighbor] = new_cost
                parent[neighbor] = node
                priority = new_cost + heuristic[neighbor]
                heapq.heappush(queue, (priority, new_cost, neighbor))

    return None, float("inf")

path, cost = a_star_search('A', 'D')
print("Path:", path if path else "No path found")
print("Total cost:", cost if path else "Not available")
```

3. AO* Search

```
# Use case: Selecting the minimum-cost solution in an AND-OR graph.
# Each list inside a node represents one alternative:
# all nodes in an alternative group must be solved (AND).
# Choosing one group among alternatives is OR.

graph = {
    'A': [['B', 'C'], ['D']],
    'B': [['E'], ['F']],
    'C': [['G']],
    'D': [['H', 'I']],
    'E': [], 'F': [], 'G': [], 'H': [], 'I': []
}

cost = {
    'A': 0, 'B': 1, 'C': 1, 'D': 2,
    'E': 2, 'F': 4, 'G': 3, 'H': 2, 'I': 2
}

def ao_star(node, memo=None):
    if memo is None:
        memo = {}

    if node in memo:
        return memo[node]

    if not graph[node]:
        memo[node] = (cost[node], [node])
        return memo[node]

    best_total = float("inf")
    best_solution = []

    for group in graph[node]:
        group_cost = cost[node]
        group_solution = [node]

        for child in group:
            child_cost, child_solution = ao_star(child, memo)
            group_cost += child_cost
            group_solution.extend(child_solution)

        if group_cost < best_total:
            best_total = group_cost
            best_solution = group_solution

    memo[node] = (best_total, best_solution)
    return memo[node]

minimum_cost, solution = ao_star('A')
print("Minimum cost:", minimum_cost)
print("Solution nodes:", solution)
```

4. Memory-Bounded Heuristic Search (IDA*)

```
# Use case: Solving a puzzle with limited memory.
```

```
# IDA* (Iterative Deepening A*) uses an f-cost threshold
# and depth-first search instead of storing a large open list.

start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def heuristic(state):
    distance = 0
    for index, value in enumerate(state):
        if value != 0:
            goal_index = goal.index(value)
            row1, col1 = divmod(index, 3)
            row2, col2 = divmod(goal_index, 3)
            distance += abs(row1 - row2) + abs(col1 - col2)
    return distance

def neighbors(state):
    zero = state.index(0)
    row, col = divmod(zero, 3)
    moves = []

    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = row + dr, col + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_index = nr * 3 + nc
            board = list(state)
            board[zero], board[new_index] = board[new_index], board[zero]
            moves.append(tuple(board))

    return moves

def ida_star(initial):
    path = [initial]
    threshold = heuristic(initial)

    def search(g, bound):
        node = path[-1]
        f = g + heuristic(node)

        if f > bound:
            return f
        if node == goal:
            return "FOUND"

        minimum = float("inf")
        for nxt in neighbors(node):
            if nxt not in path:
                path.append(nxt)
                result = search(g + 1, bound)
                if result == "FOUND":
                    return "FOUND"
                minimum = min(minimum, result)
                path.pop()

        return minimum

    while True:
        result = search(0, threshold)
        if result == "FOUND":
            return path[:]
        if result == float("inf"):
            return None
        threshold = result

solution = ida_star(start)
if solution:
    print("Solution found in", len(solution) - 1, "moves:")
    for state in solution:
        print(state[0:3])
        print(state[3:6])
        print(state[6:9])
        print()
else:
    print("No solution found")
```